

Path traversal to credential reuse

Follow-along: read one file you were never meant to read, then make it worth nothing

Overview

A program that serves a file the caller named has to decide which files the caller is allowed to ask for. The document viewer in this lab decides nothing. It takes the value of a query parameter, joins it to the end of a directory name, and hands the result to the function that opens a file. Joining is not confining: `../` in the value moves one directory towards the root before the open happens, so a caller who supplies enough of them names any file on the machine. That is **path traversal**, and it is one of the oldest bugs in web software because the fix looks like it is already there. The directory name is right at the top of the line.

Definition: Path traversal

Path traversal is a caller reaching a file outside the directory a program meant to serve from, by putting the parent-directory segment `../` into a value the program joins to a path. The joined string is not resolved and compared against the intended directory before the file is opened, so the check that would have caught it never runs. It is not a bug in the operating system, which resolves the path it was handed exactly as written.

Part 1 walks the chain that turns one file read into a database session, and every step's target is named by the step before it: `/etc/passwd` to confirm the bug, the web server's configuration to find out where the viewer's own file lives, the viewer's source to find out where its database credentials come from, and then that credential file. It ends on your own machine, running a database client against a table no request to the portal ever touches. Nothing in Part 1 is a password guess and nothing is a network attack; the whole of it is one HTTP parameter and then one login with a password that was lying on disk.

You never repair the viewer, and Part 2 changes no line of its source. What you are graded on is four changes to configuration: a rule at the web server that rejects `../`, which the same read gets past spelled differently; a limit on which directories the interpreter may open a file in at all, which stops every spelling and still cannot hide the credential file; the disclosed account bound to the address of the tier that uses it, which refuses you and stops a second job at the same time; and then one account per tier. At the end the viewer is still broken and the credential file is still readable. The password it discloses is worth nothing to you, because the database will only accept it from an address you are not at, and that is the whole lab.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

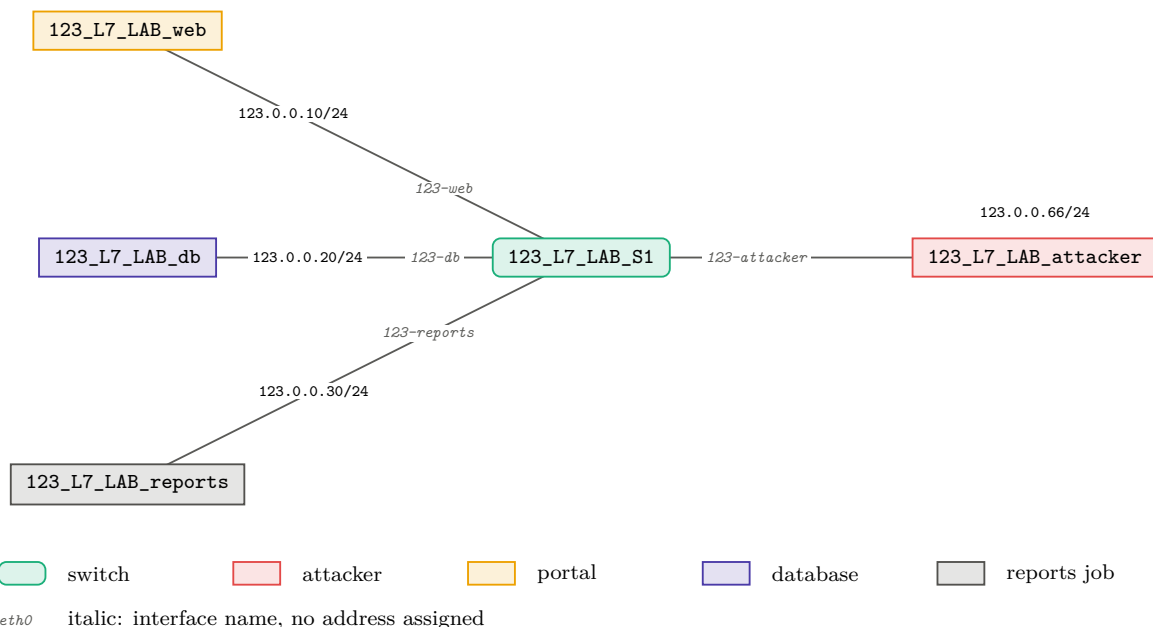
If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one category per row. Select **Software exploitation and containment**, then **Path traversal to credential reuse**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring, and loads the portal's data. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its image, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle: which files the viewer will read for you, whether the password you can read out of the portal opens a database session from your own machine, and whether the portal and the nightly report job both still work. It sends its own requests to find that out and changes nothing you are graded on. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does. Part 1 is worked on **attacker**; Part 2 is worked on **web**, then on **db**, and checked on **reports**.
- **Reset to baseline** puts all four machines back the way they were spawned, including every configuration file and every database account you changed, without tearing the lab down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

One terminal is enough for this lab; nothing here has to be left running while you send a request. Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



Four hosts on one switched segment. There is no router: every control in this lab is applied on a host, and the database sees the address of whichever host opened the connection either way.

Container	Address	Role	What runs on it
123_L7_LAB_web	123.0.0.10	portal	lighttpd, <code>view.php</code> over <code>php-cgi</code>
123_L7_LAB_db	123.0.0.20	database	MariaDB 10.11 on port 3306
123_L7_LAB_reports	123.0.0.30	reports job	<code>nightly-report</code> , run by hand here
123_L7_LAB_attacker	123.0.0.66	yours	<code>curl</code> , and the <code>mariadb</code> client

The portal serves three customer documents. The reports host runs one job overnight that counts rows in two tables the portal never reads. Both of them connect to the same database, and in the state the lab spawns in they connect as the same account.

Your machine has two tools and no others that matter: `curl`, which sends every request in Part 1, and the `mariadb` client, which opens the session at the end of it.

Assessment

There are 19 questions, placed at the step each one is about. Answer all of them in a separate document, under the same numbers. Several ask you to record a value you read off your own screen, so answer them as you go rather than from memory afterwards.

Part 1: reading files the portal never meant to serve

1A. What the viewer does with the value you give it

Definition: Document root

The **document root** is the one directory a web server maps request paths into. A request for `/a/b.txt` is served from `<document root>/a/b.txt`, and a path that resolves outside that directory is not served at all. It is the server's own boundary, and it applies to the files the server hands over itself. It says nothing about which files a program the server runs may open.

Definition: CGI

CGI is the arrangement in which a web server, instead of sending a file's bytes to the client, runs a program and sends that program's output instead. `lighttpd`'s `cgi.assign` directive maps a filename extension to the interpreter to run: with `.php` mapped to `php-cgi`, a request for `/view.php` starts `php-cgi` with that file as its script and returns what the script printed. The mapping is on the extension in the request path, so what decides between running a file and sending it is the URL, not the file.

Both of those decide what a request gets. The **document root** decides which files a request path can name at all, and **CGI** is why a request for `/view.php` runs a program instead of returning that file's text. Which of the two handled a given request is the first thing to establish, because only one of them is bounded by the document root.

Get a shell on your own machine and look at the portal.

```
# 123_L7_LAB_attacker: the site's front page, and one document through the viewer
curl -s http://123.0.0.10/
curl -s "http://123.0.0.10/view.php?file=returns-policy.txt"
```

The viewer prints a title and then the document. The title comes from the database and the body comes from a file, so one request exercises both halves of the portal.

Now ask the web server for the same document directly, without the viewer.

```
# 123_L7_LAB_attacker: two requests that do not go through the viewer
curl -s -o /dev/null -w '%{http_code}\n' "http://123.0.0.10/returns-policy.txt"
curl -s -o /dev/null -w '%{http_code}\n' "http://123.0.0.10/docs/returns-policy.txt"
```

Question 1. Record the status code each of those two requests returned. Then, for the successful request through the viewer and for either of these two, name which component opened `returns-policy.txt`: the web server itself, or a program it ran.

1B. Climbing out of the directory

The value you sent was a bare filename and the viewer found the file, so the viewer is joining your value to some directory. Nothing so far says which one. Find out by walking upwards: send a value that starts with `../`, add another one, and keep going until a file you know exists on every Linux machine comes back.

```
# 123_L7_LAB_attacker: one ../ at a time until /etc/passwd comes back
curl -s "http://123.0.0.10/view.php?file=../etc/passwd"
curl -s "http://123.0.0.10/view.php?file=../../etc/passwd"
```

When the read fails the page prints `cannot open document` under the title. That is the viewer's own text and it says nothing about why: a file that is not there and a file the program may not open produce the same line.

Question 2. Give the smallest number of `../` segments that returned the contents of `/etc/passwd`. From that number alone, state how many directories deep the directory the viewer joins your value to sits below the filesystem root.

Question 3. Predict what happens if you send six more `../` segments than the number you just found, then send them and check. Explain the result from how the kernel resolves `..` at the root of the filesystem.

1C. Reading the web server's configuration

`/etc/passwd` proves the class of bug and gives you nothing else: it is a list of accounts with no passwords in it. What is worth reading is configuration, because configuration says where everything else on the machine is. Start with the web server's own file.

```
# 123_L7_LAB_attacker: adjust the number of ../ to the one you found in 1B
curl -s "http://123.0.0.10/view.php?file=../../etc/lighttpd/portal.conf"
```

Question 4. From that file, state the absolute path of the document root and the interpreter that `.php` files are handed to. Then use the two directives you just named to explain why your request for `/returns-policy.txt` in step 1A returned the code it did.

1D. Reading the viewer's source

The document root tells you where the viewer's own file is. Ask for it.

```
# 123_L7_LAB_attacker: the viewer's file, read rather than run
curl -s "http://123.0.0.10/view.php?file=../../var/www/html/view.php"
```

Question 5. A request for `http://123.0.0.10/view.php` runs the program and returns what it printed. The request above returned the same file as text. Explain the difference by naming what opened the file in each case, and what each one used to decide.

The source shows the join, and it shows the directory being joined to.

Question 6. Give the absolute path that the successful request in step 1B opened, working it out from the directory named in the source and the value you sent. Show the joined string before it is resolved and the path after.

1E. The credential file

Near the top of the source is a `require` of a file outside the document root, and the comment above that file says no request can fetch it. Fetch it anyway.

```
# 123_L7_LAB_attacker: the file the viewer includes to get its database login
curl -s "http://123.0.0.10/view.php?file=../../etc/minilabs/db.inc.php"
```

You now hold the address of the database, an account name, and that account's password.

Question 7. The comment in that file claims that its position outside the document root means no request can fetch it. State precisely what that position does stop, and why the request you just sent was not stopped by it. Name the component whose boundary the document root is.

1F. Using the credential from your own machine

Definition: The host part of a database account

A MariaDB account is not a name. It is a pair, written `'name'@'host'`, and the host part is matched against the address the connection was opened from. `'app'@'10.0.0.5'` and `'app'@'10.0.0.6'` are two separate accounts that may hold different passwords and different privileges. The host part accepts the wildcard `%`, so `'app'@'%'` is one account that any address may use, and `'app'@'10.0.0.%'` is one account that any address on that /24 may use.

The credential file gave you a name and a password and said nothing about a **host part**, because the host part is not the client's to choose: it is recorded on the server, against the account. Open a session from your own machine and find out what it currently holds.

```
# 123_L7_LAB_attacker: substitute the account and password from step 1E
mariadb -h 123.0.0.20 -u <account> -p'<password>' -e "SELECT * FROM portal.customer"
```

Question 8. Record how many rows the `customer` table returned. Then, using the viewer's source from step 1D, state whether any request to the portal returns rows from that table, and say what in the source settles it.

Question 9. Your machine has never served a page and the database has no reason to hear from it. State what the host part of the account you are using must currently hold for both

this session and the portal's own connections to be accepted, and give one narrower value that would still accept both.

That is the whole of the attack. One query parameter, four requests, and a login.

Part 2: making the disclosure worth nothing

You are now the administrator of these four machines, and you have two facts the attacker did not. The first is that the viewer's source is not yours to change in this exercise: treat it as a vendor's program you have a support contract for and no patch from. The second is on the reports host.

```
# 123_L7_LAB_reports: the nightly job, and the credentials it uses
nightly-report
cat /etc/minilabs/report.cnf
```

It is the same account and the same password as the one on the web host. Two tiers, one credential. Nothing on this host is readable through the traversal, because the traversal only reaches files on the machine the viewer runs on; what puts this host's credential at risk is that a copy of it was on the machine the viewer runs on.

The four stages below are applied in order, and each one is checked before the next is written. Run the Status action after every stage: it prints which files the viewer will still read, whether the disclosed password still opens a session from 123.0.0.66, and whether the portal and the report job both still work. The last two lines have to stay *ok* at every stage. A change that stopped the attack by stopping the portal is not a defence.

2A. Rejecting the request at the web server

The most direct reading of Part 1 is that requests containing `../` are the problem, so the web server should refuse them. `lighttpd` can do that without any help from the viewer.

Definition: `lighttpd` conditionals and `url.access-deny`

A `lighttpd` configuration block written `$HTTP["querystring"] =~ "..."` `{ ... }` applies the directives inside it only to requests whose query string matches that regular expression. `url.access-deny` is a list of endings; a request whose URL ends with any of them is refused with 403 and nothing else runs. The empty string is an ending of every URL, so `url.access-deny = ( "" )` inside a conditional denies every request the conditional matched. The directive lives in `mod_access`, and a module that is not named in `server.modules` is not loaded: `lighttpd` then reports `url.access-deny` as an unknown config-key, ignores it, and serves the request.

What you are about to write is one **lighttpd conditional** wrapping one `url.access-deny`, which together refuse any request whose query string carries the three characters `../`.

Question 10. Before you write anything: state whether a rule that denies any request whose query string matches `../` will stop you reading `/etc/passwd` through the viewer. Give the reason your prediction rests on.

Get a shell on the portal and add the rule.

```
# 123_L7_LAB_web: load mod_access, which url.access-deny lives in
sed -i 's/( "mod_cgi" )/( "mod_access", "mod_cgi" )/' /etc/lighttpd/portal.conf
```

```
# 123_L7_LAB_web: the rule itself. The quoted RULE marker stops the shell
# expanding $HTTP before the text reaches the file.
cat >> /etc/lighttpd/portal.conf <<'RULE'

$HTTP["querystring"] =~ "\.\/" {
    url.access-deny = ( "" )
}
RULE
```

```
# 123_L7_LAB_web: parse the file, then restart the server so it is read
lighttpd -f /etc/lighttpd/portal.conf -tt
pkill -x lighttpd
rm -f /run/lighttpd.pid
lighttpd -f /etc/lighttpd/portal.conf
```

Send the request from step 1B again and record what comes back.

Definition: Percent-encoding, and where it is undone

Any character in a URL may be written as % followed by its two hexadecimal digits, so . may be written %2e and / may be written %2f. The encoded and the plain spelling name the same character; what differs is which bytes travel on the wire. A lighttpd \$HTTP["querystring"] condition is matched against the query string as it arrived, before any escape in it is undone. PHP undoes the escapes in a query parameter when it fills \$_GET, so the value the program reads is the decoded one. The condition and the program therefore see different strings for the same request.

Percent-encoding is therefore a way to name the same path in bytes the rule does not match.

Question 11. Write a request that reads /etc/passwd through the viewer with the rule in place, using the fact above. Give the exact URL, and record the HTTP status code it returned alongside the one the plain spelling now returns.

Question 12. The rule you wrote inspects a string that has not been decoded, and the program opens a path that has not been resolved. Name the two operations a check would have to perform, and in which order, for it to reject both spellings of the request; and say which component would have to hold that check.

Leave the rule in place. It is not doing nothing, and Question 18 comes back to it.

2B. Limiting which directories the interpreter may open at all

The rule failed because it inspected the request. A check that cannot be spelled around has to sit where the path is already resolved, at the point the file is opened. PHP has one.

Definition: open_basedir

`open_basedir` is a PHP configuration directive holding a colon-separated list of directories. Every function that opens a file, `file_get_contents`, `fopen`, `include` and `require` among them, refuses a path that does not resolve to somewhere under one of them. The check is applied to the path after `..` segments have been resolved, so how the path was spelled in the request does not reach it. A refused open raises a warning and returns `false`, which is the same thing a missing file returns.

`open_basedir`'s list has to hold every directory the application legitimately opens a file in, and no others. You already know all of them: the document directory from the viewer's source, the document root from the web server's configuration, and the directory holding the file the viewer includes.

```
# 123_L7_LAB_web: name the directories PHP may open a file in
echo 'open_basedir = /srv/docs/:/var/www/html:/etc/minilabs/' >> /etc/php82/php.ini
```

Nothing needs restarting. `lighttpd`'s `mod_cgi` starts a fresh `php-cgi` process for every request, and each one reads `php.ini` as it starts.

Now send all five reads from Part 1 again: `/etc/passwd` in both spellings, the web server's configuration, the viewer's source, and the credential file.

Question 13. Two of those five still return their contents. Name them, and state what those two files have in common that the other three do not.

Question 14. Removing `/etc/minilabs/` from the list would stop the credential file being read. Predict the HTTP status code and the response body an ordinary request for `returns-policy.txt` would get if you did. Name the line of the viewer's source that decides it, and say what that line does differently from the one that opens a document.

2C. Binding the account to the tier that uses it

The credential is still readable and you cannot stop that from here. What you can decide is where the database will accept it from.

Definition: The two checks MariaDB makes on a new connection

When a connection arrives, the server first looks for any account in its table whose host part could match the client's address. If there is none, it refuses with error 1130, `Host '...' is not allowed to connect to this MariaDB server`, naming the address the connection came from, before any account name or password has been considered. If there is one, it then matches the name and the password against the accounts for that address and refuses a mismatch with error 1045, `Access denied for user`. The two errors say different things: 1130 means the server has no account for that address at all, and 1045 means it has one and this is not it.

`RENAME USER` changes an account's host part and keeps its password, so after it your session is one the **host-level check** has no account to match at all.

Question 15. Before you run it: the account is about to be bound to 123.0.0.10 alone. Predict, for each of the three lines Status reports, whether it changes: the disclosed password opening a session from 123.0.0.66, the portal serving a document, and the nightly report job running.


```
# 123_L7_LAB_db: bind the shared account to the web tier's address
mariadb -u root -e "RENAME USER 'portalapp'@%' TO 'portalapp'@'123.0.0.10'; FLUSH \
PRIVILEGES;"
```

Try the session from your own machine again, and run **nightly-report** on the reports host.

Question 16. Record the error the database returned to 123.0.0.66. State which of the two errors in the box above it is, and then say which of the two you would have received instead if some other account in the server's table were still granted from '%'.

2D. One account per tier

Stage 2C refused you and stopped a job that had every right to run, because one account cannot be bound to two addresses. The state that holds all three facts at once is two accounts, each with its own password, each bound to the address of the tier that uses it. Three files change together: the database's account table, the credential file on the portal, and the credential file on the reports host.

```
# 123_L7_LAB_db: one account per tier, then drop the shared one
mariadb -u root -e "
CREATE OR REPLACE USER 'portalweb'@'123.0.0.10' IDENTIFIED BY 'pick-your-own-1';
GRANT SELECT ON portal.* TO 'portalweb'@'123.0.0.10';
CREATE OR REPLACE USER 'portalreports'@'123.0.0.30' IDENTIFIED BY 'pick-your-own-2';
GRANT SELECT ON portal.* TO 'portalreports'@'123.0.0.30';
DROP USER IF EXISTS 'portalapp'@'123.0.0.10';
FLUSH PRIVILEGES;"
```

Then put each tier's own credential on that tier. Edit `/etc/minilabs/db.inc.php` on the portal so `DB_USER` and `DB_PASS` name the first account, and `/etc/minilabs/report.cnf` on the reports host so `user` and `password` name the second. Then run **Status**: the session from 123.0.0.66 is refused, and both service lines read **ok**.

Question 17. Read the credential file through the viewer once more. It still returns a working password. State what that password is now worth to someone at 123.0.0.66 and what it is worth to the portal, and name the one property of the account that produced the difference.

Question 18. The reports host's credential was never on the machine the viewer runs on, and after this stage it is a different password from the portal's. Say what an attacker who repeats the whole of Part 1 now learns about the reports tier, and what they would have learned before this stage.

Question 19. The same refusal could be produced instead by a packet filter on the database host that drops connections to port 3306 from every address except 123.0.0.10 and 123.0.0.30. Name one thing the account's host part gives you that the packet filter does not, and one thing the packet filter gives you that the host part does not.

Checks: what "done" means

Run the **Status** action. Before Part 2, its **Reuse** block reads **ACCEPTED** and every line of its **Reach** block reads **read**. When you are done, three things hold at once:

- the database session from 123.0.0.66 is refused, and the message names that address;

- `portal serves returns-policy.txt` reads ok, so the viewer still gets its title from the database and its body from the document directory;
- `nightly report job` reads ok, so the second tier still reaches the two tables it counts.

The Reach block does not have to be empty and will not be. The viewer's source and the credential file are still readable, because the interpreter has to open both of them to serve a page, and no configuration on this machine can allow one and refuse the other. What changed is the third line.

To start over, use the Reset action. It puts all four machines back the way they spawned: the web server's configuration without your rule, `php.ini` without `open_basedir`, and the database with the one shared account granted from `'%'`.

Safety note

Every container in this lab sits on one Open vSwitch bridge with no route off the host. The traversal reaches files on 123.0.0.10 and the database session reaches 123.0.0.20, both of which this lab created and this lab removes. Nothing here is aimed at an address outside the segment, and nothing on the segment outlives the Teardown action.

Teardown

Remove the lab when you are finished: the **Teardown** action in the TUI, or `scripts/teardown.sh` run directly. It removes this lab's five containers and nothing else.